LESSONLAB

AI-Powered Lesson Creation Tool

Project Submission Report

Prepared for: AI Research Intern – Task Submission

Project type	Local AI-assisted lesson research and authoring workflow
Main flow	Research → Lesson Generation → Factual QC → Instructional QC
Technology	Python, Flask, SQLite, LLM adapters, web/book/video research
Delivery	Source code, setup guide, technical design, demo guide and this report

Purpose

LessonLab accepts a lesson title and learning objective, gathers relevant learning references, drafts a lesson intended for a 45-minute session, and generates two separate quality-control audit reports for instructor review. The application also supports instructor editing, saving, retrying failed workflow stages and exporting lesson materials.

This report documents the project design and source package. It does not claim that live API calls, external research verification or end-to-end sample runs have been completed.

1. Assignment Requirements

The supplied assignment requires research across books, websites and YouTube; structured lesson generation with explanations, examples, practical applications and knowledge checks; two independent QC layers; and review, edit, save and export capabilities.

Requirement	Implementation in LessonLab
Inputs	Lesson title and measurable learning objective.
Research	Web search, book lookup and video discovery adapters; page text extraction when available.
Lesson generation	Structured lesson draft with explanations, examples, applications, knowledge checks and 45-minute pacing.
QC Layer 1	Factual/research audit: evidence support, source reliability/relevance, contradictions and gaps.
QC Layer 2	Instructional audit: objective alignment, clarity, sequencing, examples, practice and assessment.
Review/export	Instructor review/edit/save and export of lesson, references and separate QC reports.
Error handling	Per-stage status/error tracking and retry support.

2. System Architecture

Component	Responsibility
Flask app (app.py)	Serves the instructor interface and project, retry, edit and export endpoints.
Pipeline (pipeline.py)	Coordinates research, lesson generation, factual QC and instructional QC.
Research adapters (search.py)	Searches web sources, books and videos; extracts readable page text when accessible.
LLM adapter (llm.py)	Supports Anthropic and OpenAI-compatible endpoints configured through environment variables.
SQLite store (store.py)	Persists inputs, lesson content, sources, QC outputs and edit history locally.
Exporter (exporter.py)	Produces Markdown, HTML and ZIP exports.

3. End-to-End Workflow

• Instructor enters a lesson title and learning objective.

• Research stage gathers candidate websites, book references and YouTube resources.

• Generation stage drafts the lesson with explanations, examples, applications, activities and knowledge checks, paced for 45 minutes.

• QC Layer 1 checks factual support, source reliability, contradictions and evidence gaps.

• QC Layer 2 separately checks instructional alignment, clarity, sequencing, examples and assessment.

• Instructor reviews and edits the lesson, saves changes and can rerun QC.

• Final lesson content, references and QC reports can be exported.

4. Quality Control

QC Layer 1 – Research and Factual Accuracy

Designed to flag claims without source support, questionable or irrelevant sources, contradictions, misleading statements and missing evidence. It generates a separate audit report with findings and suggested improvements.

QC Layer 2 – Instructional Quality

Designed to assess alignment with the learning objective, clarity, logical sequencing, sufficiency of explanations, examples, practical activities and knowledge checks. It generates a distinct instructional audit report.

Both audits are AI-assisted review tools. They guide instructor review and do not guarantee exhaustive factual verification.

5. Project Structure

```
lesson-creation-tool/
■■■ app.py
■■■ lessontool/ (config, exporter, llm, pipeline, search, store)
■■■ templates/index.html
■■■ static/ (app.js, style.css)
■■■ scripts/ (check_setup.py, run_samples.py)
■■■ tests/
■■■ samples/
■■■ requirements.txt
■■■ .env.example
■■■ README.md
■■■ TECHNICAL_DESIGN.md
■■■ DEMO_SCRIPT.md
■■■ SUBMISSION_REPORT.pdf
```

6. Local Setup

Windows PowerShell

```
py -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
Copy-Item .env.example .env
# Add provider credentials to .env
python app.py
```

macOS / Linux

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
# Add provider credentials to .env
python app.py
```

Open http://127.0.0.1:5000. Python 3.10+ is required. Live research needs network access; model-based generation and QC require a configured provider API key. Keep .env private and never commit API keys.

7. Sample Inputs

Required: HTTP Requests and Responses — Explain GET and POST methods and distinguish between 2xx, 4xx, and 5xx response status categories.

Additional: SQL Joins: INNER, LEFT and RIGHT — Explain how INNER, LEFT and RIGHT joins combine rows from two tables and predict result sets for small example tables.

The source package includes a script to run both sample workflows after setup. Their execution is not claimed as completed.

8. Known Limitations

• Research and page extraction are best-effort and may be affected by network restrictions, rate limits or unavailable pages.

• A search result or video link does not prove that the full source or transcript was reviewed.

• AI-generated QC reports are advisory and should be reviewed by an instructor.

• The prototype is intended for local evaluation. Public multi-user deployment would need authentication, CSRF protection, managed background jobs and stronger concurrency controls.

• Without credentials, demo mode is labelled and must not be represented as live AI-generated or fact-verified output.

9. Submission Package

The package includes application source, dependency list, environment template, README, technical design notes, demo script, sample workflow runner, tests and this PDF report.

10. Status

This is a documentation and source handover. No live provider execution, external research verification or end-to-end sample test is asserted.